

ABM BUSINESS LOGIC BIBLE

§15 — RETRY, FAILURE & RECOVERY LOGIC

V2 Section 15 · Standalone Reconciliation · Sourced From §4 v2's FAIL Domain

Enterprise systems must assume everything fails: APIs time out, AI models error, emails bounce, enrichment providers go down, CRM sync drops. V2 specifies a per-failure-type retry table. That table was already built out architecturally inside §4 v2's FAIL domain (Section 4, Integrations) because failure handling is inseparable from the integrations it protects. §15 exists as this standalone document for the same reason every other V2-numbered section does: a developer working section-by-section from V2's table of contents should find §15 here, fully specified, not buried as a sub-section of §4.

V2 failure types	6	5-Whys chains	5 (root-caused)
Rules (sourced from §4 v2)	11	New rules (this pass)	9

WHY THIS DOCUMENT EXISTS SEPARATELY FROM §4 v2

The FAIL-01 through FAIL-09 and FAIL-LEARN-01/02 rules ALREADY EXIST, built and QA'd inside the §4 v2 Integrations document. Nothing here contradicts or replaces them — every rule ID below is identical to its §4 v2 counterpart. This document's job is purely architectural bookkeeping: giving V2's own Section 15 a standalone home matching the pattern §4 through §14 have all followed, and adding the operational detail (the literal per-failure-type retry table) that V2's original text specified but §4 v2's integration-focused framing compressed into a summary matrix.

15.0 RETRY STRATEGY BY FAILURE TYPE — V2 BASELINE

Failure type	Retry strategy	Max retries	After max retries
API timeout (enrichment, CRM)	Exponential backoff: 30s, 1m, 5m, 15m	4	Dead-letter queue; manual handling within 24h
AI model failure (timeout/error)	Retry same model after 60s; if fails, try fallback model	2 primary + 1 fallback	Queue for manual message creation
Email delivery failure	No retry for hard bounces; soft bounce: retry once after 4h	1 (soft bounce only)	Mark UNDELIVERABLE; switch to LinkedIn
LinkedIn API failure	Retry after 15 minutes	3	Skip this LinkedIn touch; continue to next step
CRM sync failure	Exponential: 1m, 5m, 15m, 60m	4	Dead-letter queue; critical, manual sync required
Webhook failure (engagement)	Retry every 5 minutes for 1 hour	12	Log as missed event; flag for manual review

Rule ID	Specification
RETRY-01	Every retry attempt is logged: failure_type, attempt_number, timestamp, error_message, result. Source: V2 §15.1.
RETRY-02	Dead-letter queues are reviewed by a human at least once per business day. Source: V2 §15.1 — identical to §4 v2's general dead-letter review cadence, referenced from §12 v1 CRM-SYNC-03.
RETRY-03	If any single service has a failure rate above 20% in a 1-hour window, trigger a system alert — something is fundamentally wrong; don't keep retrying blindly. Source: V2 §15.1.

15.1 RECONCILED WITH §4 v2's FAIL DOMAIN (IDENTICAL RULE IDs)

These rules are not new — they are transcribed verbatim from §4 v2 §4.10, which generalized V2's six-row failure table into a type-matched recovery architecture covering every integration in the Bible, not only the six V2 originally named. Reproduced here under their original IDs so §15 is complete as a standalone reference.

Rule ID	Specification
FAIL-01	Recovery strategy is selected by failure type per the matrix; the API error contract (§4 v2 API-05) supplies the classification.
FAIL-02	Transient failures (timeout, 5xx, rate-limited) use exponential backoff retry with a max-attempt cap.
FAIL-03	Permanent failures (validation, auth) are not retried; they return a typed error and are logged.
FAIL-04	Stateful-integration failures (CRM sync) dead-letter with full payload for inspection and replay (links to §4 v2 INT-CRM-03, §12 v1 CRM-SYNC-01).
FAIL-05	AI generation failures retry with a fallback prompt then escalate to QC/human, never emit ungrounded output.
FAIL-06	Enrichment failures defer processing and flag contact freshness to scoring rather than blocking the pipeline.
FAIL-07	LinkedIn ban-risk and email reputation-floor breaches trigger circuit-breaker pauses (links §4 v2 INT-LIN-02 / INT-EML-04), not retries.
FAIL-08	No failure is silent: every failure is logged, counted, and alerts on repeated/threshold breach.
FAIL-09	Dead-letter and circuit-breaker states are human-reviewable and replayable; recovery actions are audited.
FAIL-LEARN-01	Every failure generates a Failure Pattern Record: root cause, precursor signals, recovery effectiveness, and prevention strategy — failures are treated as lessons, not just events.
FAIL-LEARN-02	Failure Pattern Records form a searchable organizational memory; recurring precursor patterns (e.g. signals preceding a LinkedIn ban) become predictive so the engine can pre-empt failures before they occur.

15.2 NEW BINDINGS — WHAT'S BEEN BUILT SINCE §4 v2 THAT §15 MUST NOW COVER

Nine sections have been built since §4 v2's FAIL domain was specified. Several introduced failure modes or recovery patterns of their own that §15, as the Bible's canonical failure-and-recovery reference, should explicitly absorb rather than leave scattered across their origin sections.

Rule ID	Specification
RETRY-NEW-01	§8 v2's QC-SYS-01 Failure Cluster mechanism is a §15-relevant pattern-detection layer SITTING ABOVE individual retries: where FAIL-08 logs and alerts on a single service's repeated failure, QC-SYS-01 detects clusters of QC-specific failures (narrative drift spikes, political-risk spikes) that are not retry-cluster failures but follow the same FAIL-LEARN-01 discipline — §15 is the canonical home for the general principle (FAIL-LEARN-01/02); QC-SYS-01 is its QC-specific instance.
RETRY-NEW-02	§9 v1's deliverability circuit breaker (bounce rate >5% in 7 days → pause) and §12 v2's event-ordering State Reconciliation (CRM-ORDER-03) are BOTH instances of FAIL-07's circuit-breaker pattern, generalized: a circuit breaker pauses an action category, never merely retries it. §15 names this as the general FAIL-07 pattern; §9 and §12 are its domain-specific applications.
RETRY-NEW-03	§12 v2 CRM-SYNC-02 established that a push failure must NEVER block the triggering internal state transition — this generalizes beyond CRM sync to EVERY §15 failure type: a failure in a downstream notification or sync (FAIL-04) must not cascade backward into blocking the upstream action that triggered it. This is now a standing §15 principle, not a CRM-specific one.
RETRY-NEW-04	§14 v2's STATE-GUARD-02 (a failed transition guard creates a logged Transition Rejection Event, not a silent drop) is the state-machine-layer instance of FAIL-08's no-silent-failure principle — §15 is where that principle is canonically stated; §14 v2 applies it to state transitions specifically.
RETRY-NEW-05	§13 v2's Model Drift layer (ANL-DRIFT-03, Revalidation Review is human-led, never automatic retraining) extends FAIL-LEARN-01's lesson-not-event discipline to a new failure CLASS §4 v2 didn't anticipate: a model can fail by silently becoming wrong, not only by erroring. §15 now recognizes 'silent model drift' as a failure type alongside the original six, with human-led revalidation as its recovery strategy (no retry applies).
RETRY-NEW-06	§5 v2/v3's HOT_UNENRICHED and HOT_WAITLIST states (§14 v1 STATE-ACCT-04) are themselves a FAIL-04-style dead-letter pattern applied to the account-state layer: an account that fails its enrichment or capacity-admission attempt doesn't error out of the system, it dead-letters into a named, human-reviewable holding state — exactly FAIL-09's discipline, recognized explicitly as such.
RETRY-NEW-07	§11 v2's HAND-ACC-02 four-category failure classification (reasoning / recommendation / execution / external-reality failure) is the HUMAN-DECISION-LAYER analog of §15's failure-type classification — §15 handles SYSTEM failures (API, AI, email, CRM, LinkedIn, webhook); §11 v2 handles JUDGMENT failures. Both follow the same principle (classify before you respond), applied to different layers.
RETRY-NEW-08	Every new integration or module added to the Bible after this point must, per FAIL-01's classification requirement, explicitly state which §15 failure type(s) its own failures map to (transient/permanent/stateful/generation/circuit-breaker) BEFORE it ships — §15 is hereby the canonical checklist any new §4-style domain must satisfy, closing the gap where §6 v3, §7 v2, §9 v2, §10 v2, §11 v2, §12 v2, §13 v2, and §14 v2 each independently arrived at failure-handling patterns without a single shared reference forcing consistency.
RETRY-NEW-09	§8 v1's QC-REGEN-01..04 (regeneration limits, capped at 3 standard / 2 strategy-level, then MANUAL_CREATION) is recognized explicitly as a §15 AI-generation-failure instance (FAIL-05) with its own domain-specific retry count — §15's general 'retry with fallback, then escalate' principle, applied with QC's specific numbers.

5-WHYS — WHY RETRY/FAILURE NEEDS ONE CANONICAL HOME

Why #1: Why does it matter that §15 exists as its own document when the FAIL-* rules already work fine embedded in §4 v2?

Because nine sections built AFTER §4 v2 each independently arrived at their own failure-handling pattern (§8 v2's Failure Clusters, §9's circuit breaker, §12 v2's non-blocking pushes, §13 v2's model drift, §14 v2's dead-letter-as-state) without a single document forcing them to recognize they're all instances of the SAME six original FAIL-* principles. Without §15 consolidating that, a future developer reading §13 v2 in isolation might reinvent failure-classification logic that §4 v2 already solved, under a different name, with subtly different semantics.

Why #2: Why must a failure in a downstream system never block the upstream action that triggered it (RETRY-NEW-03)?

Because making upstream correctness depend on a downstream system's availability lets a single integration's outage cascade into the engine's own internal logic being wrong, not just delayed. §12 v2 discovered this specifically for CRM pushes; §15 generalizes it because the same hazard exists for ANY notification or sync failure — INT-SLK Slack failures, §6 v3 Shared Intelligence Asset writes, anywhere a side-effect could fail without the primary action having actually gone wrong.

Why #3: Why recognize 'silent model drift' as its own failure type rather than treating it as just another AI generation failure (FAIL-05)?

Because FAIL-05's recovery (retry with fallback prompt, escalate to QC) assumes the failure is DETECTABLE at generation time — a malformed output, a rejected QC check. Model drift is invisible at generation time by definition; the model produces a plausible, QC-passing output that is simply calibrated to a world that no longer exists. Treating it as a FAIL-05 instance would apply the wrong recovery (retrying generation fixes nothing); §13 v2's human-led Revalidation Review is the only correct response, and §15 must name drift as a distinct type precisely so engineering doesn't reach for the wrong playbook.

Why #4: Why require every future integration to map its failures to §15's types BEFORE shipping, rather than letting failure handling emerge organically per module?

Because that's exactly how the Bible ended up with eight independently-invented failure patterns across §8 through §14 in the first place — each individually reasonable, collectively a maintenance burden where 'how does this module handle failure' has eight different answers instead of one classification applied eight times. A pre-ship checklist is cheap; reconciling eight divergent failure philosophies after the fact, as this document just did, is not.

Why #5: Why does §11 v2's human-judgment failure classification (reasoning/recommendation/execution/external-reality) deserve to be named as §15's sibling rather than folded directly into §15's own type list?

Because §15's types classify SYSTEM failures (an API call failed, a model errored) where the recovery is mechanical (retry, fallback, circuit-break). §11 v2's types classify JUDGMENT failures (a human or AI decision was wrong) where the recovery is recalibration, not retry. Conflating them would suggest a bad business decision can be 'retried with backoff' the way an API timeout can — a category error the two-sibling framing avoids while still showing they share one underlying principle: classify before you respond.

Root cause: failure handling needs exactly one canonical reference that every other section's failure-adjacent pattern explicitly maps onto, downstream failures must never corrupt upstream correctness, invisible failure modes like model drift need their own named type rather than being forced into a mechanical-retry category that can't fix them, and every future integration must classify against this one reference before shipping rather than inventing its own.

§15 RULE ROLL-UP & CROSS-LINKS

Group	Rules	Key cross-links
V2 retry table	RETRY-01..03	§12 v1 CRM-SYNC-03 (shared DLQ review cadence)
§4 v2 FAIL domain (reconciled)	FAIL-01..09, FAIL-LEARN-01..02	Identical IDs to §4 v2 §4.10 — no semantic change, standalone placement only
New bindings (this pass)	RETRY-NEW-01..09	§8 v2 QC-SYS-01, §9 v1 deliverability breaker, §11 v2 HAND-ACC-02, §12 v2 CRM-SYNC-02/ORDER-03, §13 v2 ANL-DRIFT-03, §14 v1/v2 dead-letter states

§15 COMPLETE — WHAT IT ESTABLISHED

§15 now exists as its own standalone document, matching every other V2-numbered section's pattern. Its content is NOT new business logic at the integration level — the eleven §4 v2 FAIL-* rules are transcribed verbatim, unchanged, because they were already correctly specified. What §15 adds is the missing consolidation: nine rules recognizing that §8 through §14 each independently built a failure-handling pattern that is actually an instance of one of §4 v2's original six failure types, plus one genuinely new type (silent model drift, from §13 v2) that none of the original six could correctly classify. §15 is now the canonical reference every future integration must map against before shipping.

With this document, §15 moves from 'covered implicitly inside §4 v2' to a real, citable, standalone section — closing the one structural gap in the Placement Map that wasn't a content gap but a findability gap.

PLACEMENT MAP UPDATE

§15 status: confirmed Complete, now WITH a standalone artifact (previously Complete-but-embedded-in-§4). No open questions — this was reconciliation, not new design.

ON THE HORIZON

§17 KSA-Specific Business Rules remains the Bible's only fully 'Not started' section: Sunday–Thursday work week, Ramadan/Eid blackouts, SAMA CSF, PDPL, Islamic banking terminology, Vision 2030 alignment. §16 Timing Matrix consolidation is the natural lighter companion to bring the Bible to full V2 coverage.